Programming for Problem Solving Using Python
Unit-1

Session-5: Python- Understanding Variables, Numbers

Iflah Aijaz

School of Engineering & Technology (SOET)
K. R. Managalam University
Program: BTech (CSE)/BTech Full Stack Xebia/BTech UX UI/
B.Sc. DS/B.Sc. Cyber/MC

2025

Session-4: RECAP

- **Variables and Data Types**
  - Numbers: `int`, `float`, complex
  - Strings: `str`
  - Lists: `list`
  - Tuples: `tuple`
  - Dictionaries: `dict`
  - Booleans: `bool`
- **Control Flow**
  - Conditional Statements: `if`, `elif`, `else`
  - Loops: `for`, `while`, `break`, `continue`, `pass`
- **Functions and Classes**
  - Function Definition: `def`
  - Lambda Functions: `lambda`

Session-4: RECAP

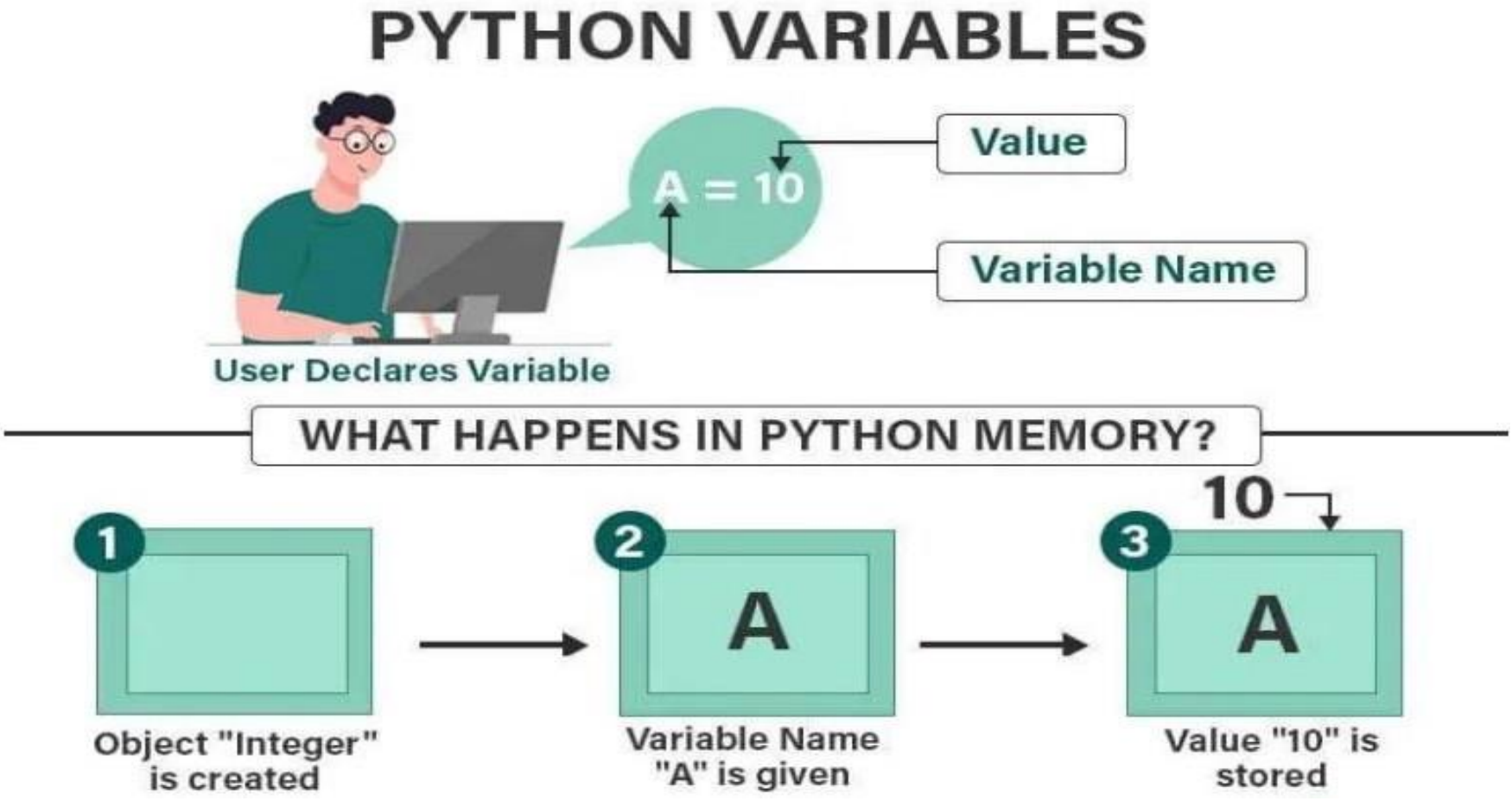
- Class Definition: `class`
- Object Creation: `self`, `init`

- **Modules and Packages**
  - Importing Modules: `import`, `from`
- **Error Handling**
  - Exceptions: `try`, `except`, `finally`, `raise`
- **Common Keywords**
  - `None`, `True`, `False`
  - `and`, `or`, `not`, `is`, `in`
  - `return`, `yield`, `global`, `nonlocal`

Session-05

Python- Understanding Variables, Numbers

Understanding Variables in Python



Understanding Variables in Python

–> A variable in programming is used to store information that you want to re-use in your code.
–> A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

```
# Variable assignment
x = 5
name = "John"
is_student = True

# Variable reassignment
x = 10
```

Understanding Variables in Python (contd.)

When naming variables in Python, there are a few rules that you need to follow:

- Variable names can only contain letters, numbers, and underscores.
- Variable names cannot start with a number.
- Variable names are case sensitive (my_variable is different from My_Variable).

Understanding Variables in Python (contd..)

There are also a few conventions that people follow when naming variables:

- Variable names should be short but descriptive.
- Variable names should be all lowercase.
- If you need to use multiple words in your variable name, you can separate them with underscores. Example: my_very_long_variable_name = 42.

Understanding Variables in Python (contd...)

Remember to be as clear and concise as possible when naming variables. This will help make your code easier to read and understand.

Some examples of good variable names:

1. my_variable
2. var_name
3. num_of_items
4. user_input

Understanding Variables in Python (contd...)

Declaring Variables

In Python, we need not declare a variable with some specific data type.

Python has no command for declaring a variable. A variable is created when some value is assigned to it. The value assigned to a variable determines the data type of that variable.

Thus, declaring a variable in Python is very simple.

Understanding Variables in Python (contd...)

Declare an integer variable.

- Name the variable
- Assign an integer value to it

Example:-

```
x=2
print(x)
print(type(x))
```

```
2
<class 'int'>
```

Understanding Variables in Python (contd...)

Declare a string variable.
Assign a string value to the variable and it will become a string variable. In Python, the string value cane be assigned in single quotes or double quotes.

Example:-

```
x='2'
print(x)
print(type(x))
```

```
2
<class 'str'>
```

Understanding Variables in Python (contd...)

Declare a float variable.
A float variable can be declared by assigning float value. Another way is by typecasting. We will use both.

Example:-

```
x=2.0
print(x)
y=float(2)
print(y)
```

```
2.0
2.0
```

Understanding Variables in Python

Python Variable Types

Python variable types, also known as variable data types, show the type of data stored in the variable.

Variable Types	Basic Examples	Real World Examples
Integer	Z = 1	Release_Year= 2003
Float	Z = 1.23	Rating = 8.2
String	Z = "Python"	Movie = "Finding Nemo"
Boolean	Z = True	Is_Released = True
List	Z = ["Alice", "Lily"]	Actors = ["Elen DeGeneres", "Alexander Gould"]
Dictionary	Z = {"Name": "Alan", "ID": 1}	Box_Office_Collection= {"US": "339.7M", "UK": "67.1M"}
Tuple	Z= ("Monday", 26, "June")	Release_Date = ("30th", "May", 2003)
Set	Z= {2,3,5}	Genre = {"Animation", "Family", "Adventure"}

Understanding Variables in Python (contd...)

- Just name the variable
- Assign the required value to it
- The data type of the variable will be automatically determined from the value assigned, we need not define it explicitly.

Understanding Variables in Python (contd...)

Working with Python Variables

- Variable Declaration: Giving a value to a new variable
- Variable Assignment: Giving a new value to an already declared variable
- Variable Manipulation: Manipulating a variable for a specific result
- Printing the Variable: Displaying the result value of a variable
- Variable Deletion: Removing a variable from memory

Understanding Variables in Python (contd...)

Let us explore each stage in detail using the following code as an example.

```
# Variable Declaration
Location = "Switzerland"
Location, Flight_Number = "Switzerland", "GEN300"
Location = Destination = "Switzerland"
# Variable Assignment
Location = "New York"
# Variable Manipulation
Tourist_spot = Location + " Empire State Building"
# Printing the Variable
print("Destination is",Location)
print("I am going to",Location,"to visit",Tourist_spot)
# Deleting the Variable
del Location
print("Location variable is deleted")
print(Location)
```

figures/KRMU-

Understanding Variables in Python (contd...)

Local and Global Variables in Python

- When working with variables in Python, you need to be aware of the scope of the variable. The scope of a variable is the part of your code where the variable is available. When you create a variable in Python, it is either a local or global variable.
- A local variable is only available within the scope in which it is created. For example, if you create a variable inside of a function, it will only be available inside of that function.

Understanding Variables in Python (contd...)

Local and Global Variables in Python

- A global variable is available throughout your entire Python program. For example, if you create a variable at the top level of your program (outside of any functions), it will be globally available throughout your program.
- Simply stated, if you wish to use the same variable throughout your program dedare it as a global variable; whereas, if you want to utilize that variable in only one function or procedure of your code, dedare it as a local variable.

Understanding Variables in Python (contd...)

Below is an example of a global variable:

Python

```python
my_var = "Hello, world!"

def my_func():
    print(my_var)
```

In this example, the variable my_var is declared as a global variable. This means that it can be accessed from anywhere in your program. In the function my_func(), we simply print the value of my_var to the screen.

Understanding Variables in Python (contd....)

If we wanted to create a local variable inside of the function instead:

Python

```python
def my_func():
    my_var = "Hello, world!"
    print(my_var)
```

In this example, the variable my_var is declared as a local variable inside of the function. This means that it can only be accessed from within the function. If we try to access it outside of the function, we will get an error.

Understanding Variables in Python (contd....)

Using the **global keyword**, you can also declare variables as global inside of a function. Take a look at the following example:

Python

```python
def my_func():
    global my_var
    my_var = "Hello, world!"
    print(my_var)
```

In this code, we use the global keyword to declare the variable my_var as a global variable inside of the function. This means that we can access it from anywhere in our program, even outside of the function.

Understanding Numbers in Python

```python
# Integer
x = 5

# Float
y = 3.14

# Arithmetic operations
sum = x + y
difference = x - y
product = x * y
quotient = x / y
```

Temperature conversion using variables and numbers

```python
# Variables representing temperatures
celsius = 20

# Celsius to Fahrenheit conversion
fahrenheit = (celsius * 9/5) + 32
print("Celsius:", celsius, " C ")
print("Fahrenheit:", fahrenheit, " F ")

# Fahrenheit to Celsius conversion
fahrenheit = 68
celsius = (fahrenheit - 32) * 5/9
print("\nFahrenheit:", fahrenheit, " F ")
print("Celsius:", celsius, " C ")
```

Calculate simple interest using variables and numbers

```
# Variables representing principal amount,
rate of interest, and time period
principal = 1000
rate = 5
time = 2
# Calculating simple interest
simple_interest = (principal * rate * time)/100

# Displaying the result
print("Principal Amount:", principal)
print("Rate of Interest:", rate, "%")
print("Time Period:", time, "years")

```

Test Your Knowledge- Fill in the Blanks

1. A variable in Python is a _____that stores data.
2. Variables in Python can store various types of data, such as _____, _____, _____, and _____.
3. In Python, variables do not need to be _____ with a specific data type.
4. Integer variables in Python can hold whole numbers such as _____and _____.

Test Your Knowledge- Fill in the Blanks

5. Floating-point variables in Python can hold numbers with _____.
6. When performing arithmetic operations in Python, integer division using "//" returns the _____of the division.
7. The modulus operator "%" returns the _____of a division operation.
8. Python provides built-in functions such as _____ and _____for converting variables between different data types.

Test Your Knowledge- Fill in the Blanks (Solutions)

1. A variable in Python is a container that stores data.
2. Variables in Python can store various types of data, such as integers, floats, strings, and booleans.
3. In Python, variables do not need to be declared with a specific data type.
4. Integer variables in Python can hold whole numbers such as 5 and -10.
5. Floating-point variables in Python can hold numbers with decimals.

Test Your Knowledge- Fill in the Blanks (Solutions)

6. When performing arithmetic operations in Python, integer division using "//" returns the quotient of the division.
7. The modulus operator "%" returns the remainder of a division operation.
8. Python provides built-in functions such as int() and float() for converting variables between different data types.

Test Your Knowledge- MCQ

1. Which of the following is a valid way to comment out a single line in Python?

   a. // This is a comment
   b. # This is a comment
   c. /* This is a comment */
   d. <! – – This is a comment – – >

2. Which of the following is a correct way to initialize multiple variables in a single line in Python?

   a. a, b = 10, 20, 30
   b. a = b = c = 10
   c. a = 10, b = 20
   d. a = 10; b = 20

Test Your Knowledge- MCQ

3. What will be the output of the following code snippet?

```
x = 10
y = 3
result = x // y
print(result)
```

a. 3.3
b. 3.0
c. 3.33
d. 3, without the decimal part

Test Your Knowledge- MCQ

4. Consider the following code snippet:

```
a = 5
b = 2
c = a ** b
```

What will be the value of ’c’?

a. 7
b. 10
c. 25
d. 32

Test Your Knowledge- MCQ (Solutions)

1. b. # This is a comment
2. b. a = b = c = 10
3. d. 3, without the decimal part
4. c. 25

Test Your Knowledge- Interview Questions

1. What are variables in Python and how are they declared?
2. Explain the difference between integers and floating-point numbers in Python.
3. How do you perform type conversion in Python? Provide examples.
4. Discuss the concept of dynamic typing in Python and its advantages.
5. What are the arithmetic operators available in Python? Give examples of each.

Test Your Knowledge- Interview Questions (Solutions)

1. Ans:- Variables in Python are used to store data.They are declared by assigning a value to a name using the "=" operator. Example: `x = 10`
2. Ans:- Integers are whole numbers (e.g., 5, -3). Floating-point numbers (floats) include a decimal point or use scientific notation (e.g., 3.14, 1.2e3).
3. Ans:- Type conversion can be done using built-in functions like `int()`, `float()`, `str()`, etc. Example: `x = int("10")`

Test Your Knowledge- Interview Questions (Solutions)

4. Ans:- Python is dynamically typed, meaning you don't need to declare the type of a variable before assigning a value to it. This flexibility allows for faster development and easier code maintenance.
5. Ans:- Arithmetic operators include addition (+), subtraction (-), multiplication (*), division (/), integer division (//), modulus (%), and exponentiation (**). Example: `result = 10 // 3` (Integer division)

Thank You!

Any Questions?